WEEK 2 · LESSON 9 OF 24 · HANDOUT

Overfitting: Dropout, Weight Decay and Early Stopping

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=mFZazxxCKbw — open the lesson online to watch inside the course.

🎓 **Go deeper:** MIT 6.S191: Introduction to Deep Learning (2024) — https://www.youtube.com/watch?v=ErnWZxJovaM

Your goal in this lesson: Recognise overfitting in learning curves and fight it with dropout, weight decay and early stopping.

Big networks memorise

Neural networks have many parameters, so with little data they can simply **memorise** the training examples — including their noise. The signs are the same as in Machine Learning Fundamentals: training results keep improving while validation results get worse.

To make overfitting easy to see, we train a large network (more than 80,000 weights) on only **200** digit images:

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

digits = load_digits()
X, y = digits.data / 16.0, digits.target
X_rest, X_val, y_rest, y_val = train_test_split(X, y, test_size=0.5, random_state=0,
stratify=y)
X_small, _, y_small, _ = train_test_split(X_rest, y_rest, train_size=200, random_state=0,
stratify=y_rest)
Xs, ys = torch.tensor(X_small, dtype=torch.float32), torch.tensor(y_small)
Xv, yv = torch.tensor(X_val, dtype=torch.float32), torch.tensor(y_val)

def big_net(dropout=0.0):
    return nn.Sequential(
        nn.Linear(64, 256), nn.ReLU(), nn.Dropout(dropout),
        nn.Linear(256, 256), nn.ReLU(), nn.Dropout(dropout),
        nn.Linear(256, 10))

def train(dropout=0.0, weight_decay=0.0, epochs=300, seed=0):
    torch.manual_seed(seed)
    model = big_net(dropout)
    opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=weight_decay)
    train_loss, val_loss = [], []
    for epoch in range(epochs):
        model.train()
        perm = torch.randperm(len(Xs))
        for start in range(0, len(perm), 32):
            idx = perm[start:start + 32]
            opt.zero_grad()
            F.cross_entropy(model(Xs[idx]), ys[idx]).backward()
            opt.step()
        model.eval()
        with torch.no_grad():
            train_loss.append(F.cross_entropy(model(Xs), ys).item())
            val_loss.append(F.cross_entropy(model(Xv), yv).item())
    with torch.no_grad():
        val_acc = (model(Xv).argmax(1) == yv).float().mean().item()
    return train_loss, val_loss, val_acc

tr, va, acc = train()
best = int(np.argmin(va))
print(f"best validation loss {min(va):.3f} at epoch {best}; at the end {va[-1]:.3f}")
print("final training loss:", round(tr[-1], 4), " validation accuracy:", round(acc, 3))
plt.plot(tr, label="training loss")
plt.plot(va, label="validation loss")
plt.axvline(best, color="grey", ls="--")
plt.legend()
plt.show()
```

The training loss falls to almost **zero** — the network has memorised the 200 images. The validation loss is lowest after roughly **30–40 epochs** and then **rises** again as the network becomes over-confident on what it memorised.

Tool 1 — Early stopping

The simplest cure: stop when the validation loss stops improving, and keep the **best** weights.

```
import copy

def train_early_stopping(patience=20, max_epochs=300, seed=0):
    torch.manual_seed(seed)
    model = big_net()
    opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=0.0)
    best_loss, best_state, waited = float("inf"), None, 0
    for epoch in range(max_epochs):
        model.train()
        perm = torch.randperm(len(Xs))
        for start in range(0, len(perm), 32):
            idx = perm[start:start + 32]
            opt.zero_grad()
            F.cross_entropy(model(Xs[idx]), ys[idx]).backward()
            opt.step()
        model.eval()
        with torch.no_grad():
            v = F.cross_entropy(model(Xv), yv).item()
        if v < best_loss:
            best_loss, best_state, waited = v, copy.deepcopy(model.state_dict()), 0
        else:
            waited += 1
            if waited >= patience:
                print("stopped at epoch", epoch)
                break
    model.load_state_dict(best_state)
    return model, best_loss

model, best_loss = train_early_stopping()
print("best validation loss kept:", round(best_loss, 3))
```

Patience is how many epochs to wait for an improvement before giving up. Early stopping also saves a lot of training time.

Tool 2 — Dropout

Dropout switches off a random share of neurons (for example 50%) at every training step. No neuron can rely on one particular partner, so the network learns more robust, spread-out features — like a football team that trains with random players missing. At prediction time (`model.eval()`) all neurons are used again.

```
layer = nn.Dropout(0.5)
x = torch.ones(1, 8)
layer.train(); print("training:", layer(x))
layer.eval();  print("evaluating:", layer(x))
```

During training about half the values become 0 and the rest are doubled (so the average stays the same). In eval mode nothing changes.

Tool 3 — Weight decay

Weight decay shrinks every weight a little at each step (the `weight_decay` setting of AdamW) — the same idea as Ridge regression. Smaller weights mean a smoother, simpler function.

Compare them

```
for name, kwargs in [("no regularisation", {}),
                     ("dropout 0.5", {"dropout": 0.5}),
                     ("weight decay 1e-3", {"weight_decay": 1e-3}),
                     ("dropout + weight decay", {"dropout": 0.5, "weight_decay": 1e-3})]:
    tr, va, acc = train(**kwargs)
    print(f"{name:24s} best val loss {min(va):.3f}  final val loss {va[-1]:.3f}  val acc
{acc:.3f}")
```

Dropout clearly lowers the validation loss. Weight decay of 0.001 changes little on its own here (try 0.01), and dropout plus weight decay ends with the lowest validation loss and the best accuracy. The accuracy differences are small here because the digits are easy — on harder data, regularisation often decides whether a model is usable at all.

The most powerful cure: more data

Regularisation helps, but nothing beats **more and more varied training data**. When collecting more is impossible, you can create variations of the data you have — **data augmentation**, Lesson 15.

Key terms

- **Overfitting** — memorising training data instead of learning general patterns.
- **Early stopping** — stop when validation loss stops improving and keep the best weights.
- **Patience** — epochs to wait for improvement before stopping.
- **Dropout** — randomly switching off neurons during training.
- **Weight decay** — shrinking weights a little at each step.

Remember this: Watch the validation curve: early stopping, dropout and weight decay keep big networks from memorising, and more data is the strongest cure of all.

Name: _____ Date: _____

Activity

Task 1

Train with 800 training images instead of 200. Does the network still overfit as badly?

Task 2

Try dropout rates 0.2, 0.5 and 0.8. Which is too much?

Task 3

Change patience to 5 and to 50. How do the stopping epoch and the best loss change?

Task 4

Plot the training and validation loss for "dropout + weight decay" on the same chart as "no regularisation".

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. Training loss is near 0 but validation loss is rising. What is happening?
   A. Overfitting
   B. The learning rate is zero
   C. The data is unlabelled
   D. Underfitting
2. What does early stopping keep?
   A. Only the biases
   B. The last weights
   C. Random weights
   D. The weights from the epoch with the best validation loss
3. What does dropout do during training?
   A. Removes layers forever
   B. Deletes training examples
   C. Lowers the learning rate
   D. Randomly switches off a share of neurons at each step
4. What happens to dropout in model.eval() mode?
   A. It drops even more neurons
   B. It halves every output
   C. It crashes
   D. It is switched off and all neurons are used
5. Weight decay is most similar to which classic method?
   A. PCA
   B. Ridge (L2) regularisation
   C. Decision trees
   D. k-means
6. What is usually the strongest cure for overfitting?
   A. A bigger learning rate
   B. Training longer
   C. Fewer validation examples
   D. More and more varied training data

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.